

SQL-запросы, связи и индексы

Читать и писать основные SQL-запросы: SELECT, WHERE, ORDER BY, JOIN, INSERT, UPDATE, DELETE и понимать роль индекса/транзакции.

DAY 22

QUERY

SELECT

WHERE

JOIN

index

ЦЕЛЬ ДНЯ

Что ты сможешь после этого дня

Конкретные действия вместо списка терминов.

- 01 Читать и писать основные SQL-запросы: SELECT, WHERE, ORDER BY, JOIN, INSERT, UPDATE, DELETE и понимать роль индекса/транзакции.
- 02 Объяснить минимум 5 терминов темы своими словами.
- 03 Прочитать показанный пример записи и предсказать результат.
- 04 Самостоятельно решить похожую задачу после разбора.

СВЯЗЬ

Новая тема не появляется из ниоткуда

Связываем сегодняшний шаг с уже построенной картой курса.

ВЧЕРА

Базы данных и SQL

Что уже есть в нашей модели.



СЕГОДНЯ

SQL-запросы, связи и индексы

Вчера: Базы данных и SQL. Сегодня мы добавляем следующий слой: sql-запросы, связи и индексы.

ЗАЧЕМ

Зачем мне это?

Каждая тема решает конкретную проблему.

ПРОБЛЕМА → ПОЛЬЗА

Структура базы бесполезна без умения задавать вопросы данным. SQL — язык, которым приложение разговаривает с реляционной БД.

Вопрос себе

Какую реальную проблему решает сегодняшняя тема?

Ответь до следующего слайда.

Сначала — человеческий образ

Потом обязательно связываем его с настоящей техникой.

БЫТОВОЙ ОБРАЗ

SQL-запрос похож на точную инструкцию библиотекарю: что показать, из какой полки, по какому условию и в каком порядке.

ПЕРЕВОД В IT

query	формальный запрос к базе данных
SELECT	SQL-команда чтения данных
WHERE	условие фильтрации строк

Слой 1: query • SELECT • WHERE

Сначала смысл, затем профессиональный термин.

**query**

формальный запрос к базе данных

**SELECT**

SQL-команда чтения данных

**WHERE**

условие фильтрации строк

Слой 2: JOIN • index • transaction

Сначала смысл, затем профессиональный термин.

 JOIN

операция соединения связанных таблиц

 index

дополнительная структура для ускорения поиска

 transaction

группа операций с принципом всё или ничего

Слой 3: INSERT • UPDATE

Сначала смысл, затем профессиональный термин.

**INSERT**

SQL-команда добавления новых строк

**UPDATE**

SQL-команда изменения существующих строк

Не заучивай формулировку. Придумай свой пример использования каждого термина.

Слой 4: DELETE • rollback

Сначала смысл, затем профессиональный термин.

**DELETE**

SQL-команда удаления строк

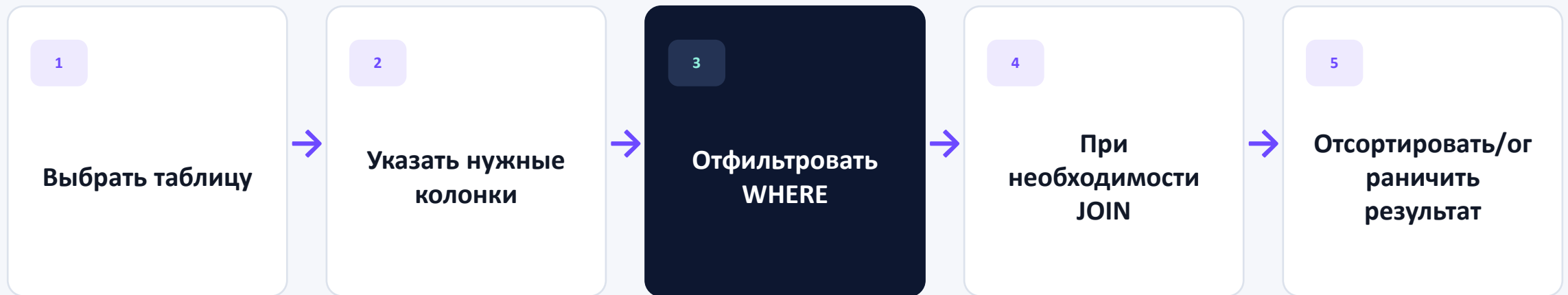
**rollback**

отмена изменений незавершённой транзакции

Не заучивай формулировку. Придумай свой пример использования каждого термина.

Как это работает по шагам

Проследи движение слева направо.



Проговори эту цепочку своими словами, не подсматривая в текст.

Первый минимальный пример

Смотрим только на одну основную идею.



day22_step1

```
SELECT title, days  
FROM courses  
WHERE days = 30  
ORDER BY title;
```

ЧТО ИСКАТЬ

- Где данные?
- Где действие?
- Где результат?
- Что изменится, если заменить одно значение?

Разбираем пример построчно

Код не должен оставаться магической конструкцией.

SELECT title, days

SELECT определяет, какие данные хотим получить из БД.

FROM courses

Python выполняет эту строку сверху вниз.

WHERE days = 30

Сохраняем значение или результат вычисления под понятным именем.

ORDER BY title;

Python выполняет эту строку сверху вниз.

РЕЗУЛЬТАТ

Код → выполнение → результат

Всегда связываем запись с тем, что реально получилось.



до запуска

```
SELECT title, days  
FROM courses  
WHERE days = 30  
ORDER BY title;
```

РЕЗУЛЬТАТ

Только 30-дневные курсы,
отсортированные по title

ИЗМЕНИЛИ

Теперь изменим пример

Сначала предскажи результат. Не запускай код сразу.

```
SELECT u.name, t.title  
FROM users AS u  
JOIN tasks AS t ON t.user_id = u.id  
WHERE t.done = FALSE;
```

changed



**Что появится в
результате?**

Сформулируй ответ словами.

ОТВЕТ

Сверяем прогноз

Не просто смотри ответ — объясни, почему он именно такой.

РЕЗУЛЬТАТ

Имена пользователей и их незавершённые задачи

ПОЧЕМУ

Python/система выполняет шаги в записанном порядке. Изменённые данные или действие дают новый результат.

Сравни с первым примером: что изменилось?

Как писать неправильно

Ошибка — отдельный учебный материал, а не повод её скрывать.



```
SELECT * FROM courses  
WHERE days = 30 AND  
ORDER BY title;
```

wrong

СТОП

**Что здесь может сломаться
или ввести нас в
заблуждение?**

Сначала найди проблему сам.

Почему возникает проблема

Переводим техническую причину на обычный язык.

После AND должно быть ещё одно условие. ORDER BY уже другая часть запроса, поэтому синтаксис нарушен.

Алгоритм отладки: симптом → конкретная строка/шаг → причина → минимальное исправление → повторная проверка.

Исправленный вариант

Меняем только то, что действительно было причиной.



fixed

```
SELECT * FROM courses  
WHERE days = 30  
ORDER BY title;
```

ПРАВИЛО

**Не переписывай всё
решение. Найди
минимальное
осмысленное
исправление.**

Что изменилось после исправления?

Ответь без запуска.

- 1 Исчезла ли исходная причина проблемы?
- 2 Какой результат/поведение теперь ожидаем?
- 3 Как проверить это одним маленьким примером?

Сначала увидим полностью разобранный пример

Только после него будет самостоятельная практика.

ЗАДАЧА

Показать имена пользователей и названия только выполненных задач.

Не начинаем с кода. Сначала переводим задачу на человеческий язык.

КАК ДУМАТЬ

Как рассуждает программист

Разбиваем большую формулировку на маленькие действия.

- 1 **Нужны данные из `users` и `tasks`.**
- 2 Связываем таблицы по `user_id`.
- 3 Фильтруем `done = TRUE`.
- 4 Выбираем только нужные колонки.
- 5 Проверяем, что JOIN не создаёт лишних сочетаний.

Шаг 1: собираем основу

Не показываем 20 строк сразу — строим решение частями.



solution_part1

```
SELECT u.name, t.title  
FROM users AS u  
JOIN tasks AS t ON t.user_id = u.id
```

Проверь

- Какие данные уже есть?
- Какой шаг плана реализован?
- Что ещё осталось добавить?

ИТОГОВЫЙ КОД

Шаг 2: итоговое решение

Теперь видим программу/запрос целиком.



```
SELECT u.name, t.title  
FROM users AS u  
JOIN tasks AS t ON t.user_id = u.id  
WHERE t.done = TRUE  
ORDER BY u.name;
```

solution_final

Проверяем результат

Сравниваем то, что ожидали, с тем, что получили.

РЕЗУЛЬТАТ

Одна строка на каждую выполненную задачу с именем владельца

ГЛАВНОЕ

Задача была решена не потому, что мы «знали готовый код», а потому что сначала построили понятный план.

ПОПРОБУЙ САМ

Практика 1

Задача очень похожа на разобранную — но теперь первый ход за тобой.

Выведи пять самых новых задач, если есть колонка `created_at`.

Сначала выпиши 2–5 шагов решения словами.

ПОДСКАЗКА

Подсказка — только направление

Попробуй ещё раз до просмотра готового решения.

ORDER BY created_at DESC и LIMIT 5.

Вопрос себе: какие конструкции сегодняшнего дня нужны, а какие пока лишние?

Один из правильных вариантов

Сравни логику, а не количество строк.



practice_solution

```
SELECT id, title, created_at  
FROM tasks  
ORDER BY created_at DESC  
LIMIT 5;
```

Ожидаемый результат

Если твой код устроен иначе, но даёт правильное и понятное решение — это нормально.

РЕЗУЛЬТАТ

Не более пяти задач, самые новые сверху

Сравни

- Результат
- Понятность имён
- Лишние шаги
- Обработку ошибок

Практика 2 — мини-проект

Соединяем несколько элементов дня в одну маленькую законченную задачу.

ЗАДАНИЕ

Обнови задачу id=7 как выполненную, но сначала сформулируй безопасный WHERE.

Попробуй сначала без подсказки. Разрешено подсматривать в сегодняшние примеры.

Разбор мини-проекта

Вот один рабочий путь.



mini_project

```
UPDATE tasks  
SET done = TRUE  
WHERE id = 7;
```

РЕЗУЛЬТАТ ПРОЕКТА

Что должно получиться

Связываем итог с задачей и проверяем смысл.

РЕЗУЛЬТАТ

Изменяется только задача с id=7

Объясни каждую важную строку/команду своими словами. Если не можешь — вернись к конкретному месту, а не перечитывай весь день.

Четыре ловушки новичка

Лучше увидеть их здесь, чем случайно закрепить как привычку.

01 **Делать UPDATE/DELETE без WHERE.**

02 **Использовать SELECT * без нужды.**

03 **Создавать индексы на каждую колонку.**

04 **Считать JOIN «склеиванием по порядку строк», а не по условию.**

Проверь себя: 1–2

Не листай дальше, пока не сформулируешь ответ.

1

Что такое «query» и какую проблему это понятие помогает описать?

2

Чем «query» отличается от «JOIN»? Объясни без копирования определения.

Проверь себя: 3–4

Здесь проверяем понимание кода и процесса мышления.

- 3 Какой результат даёт первый минимальный пример и почему?
- 4 Какую ошибку/риск мы разобрали сегодня и какое минимальное исправление нужно?

ОТВЕТЫ

Ответы к вопросам 1–2

Формулировка может отличаться — важен смысл.

1 query — формальный запрос к базе данных.

2 query: формальный запрос к базе данных. JOIN: операция соединения связанных таблиц.

ОТВЕТЫ

Ответы к вопросам 3–4

Сверяем причинно-следственную связь.

3 Только 30-дневные курсы, отсортированные по title

4 После AND должно быть ещё одно условие. ORDER BY уже другая часть запроса, поэтому синтаксис нарушен.

ИТОГ

Сегодня мы научились

Не перечисляем слова — фиксируем новые способности.

- ✓ Читать и писать основные SQL-запросы: SELECT, WHERE, ORDER BY, JOIN, INSERT, UPDATE, DELETE и понимать роль индекса/транзакции.
- ✓ Читать и объяснять пример темы «SQL-запросы, связи и индексы».
- ✓ Находить типичную ошибку и формулировать причину.
- ✓ Решать похожую задачу по шагам, а не угадывать готовый код.

ДАЛЬШЕ

Что будет завтра

Связываем сегодняшний результат со следующим шагом.

API, REST и JSON

Завтра: API, REST и JSON. Сегодняшняя модель станет опорой для следующего шага.

Сегодняшняя тема не заканчивается здесь — она станет инструментом в следующем дне.